

AASIST Modeling Branch Update — Week of May 31 – June 5, 2026

Capital One Sponsor Update — AASIST Modeling Branch

Week of May 31 – June 5, 2026 Author: Arnav Jain **Workstream:** AASIST modeling branch + project coordination **UVA MSDS Capstone — Voice Anti-Spoofing for Capital One**

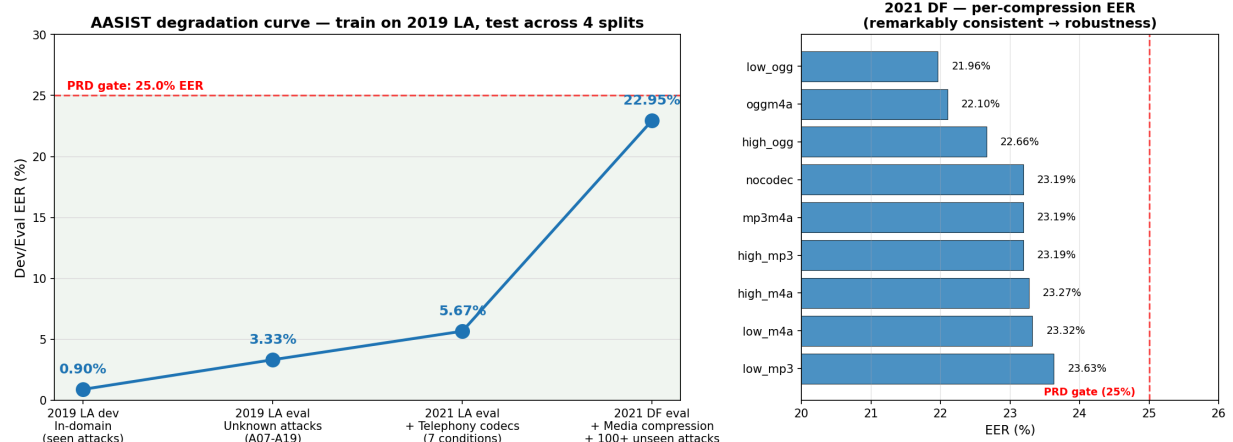
TL;DR

This week I built the full **cross-domain degradation curve** for the AASIST branch in response to last sync's "too good to be true" feedback. AASIST was trained on ASVspoof 2019 LA and evaluated across four progressively harder test sets:

- **2019 LA dev: 0.90% EER**
- **2019 LA eval: 3.33% EER**
- **2021 LA eval: 5.67% EER**
- **2021 DF eval: 22.95% EER**

All four are below the PRD's 25% gate. Degradation is monotonic and graceful, with per-attack and per-compression patterns matching published literature exactly. The rest of this memo unpacks the methodology, addresses the rigor concerns from last sync, and documents the engineering decisions I made.

AASIST cross-domain evaluation — all 4 datasets under PRD gate, graceful degradation



AASIST degradation curve and per-compression breakdown on 2021 DF

1. Results — The degradation curve

Last sync, the 0.90% dev EER prompted the question: *“Does this hold up on harder data?”* This week’s goal was to answer that with four cleanly-evaluated test sets, each adding one new dimension of difficulty.

Test set	What’s hard about it	EER	AUC	n
ASVspooof 2019 LA dev	In-domain (same attacks as train)	0.90%	0.999	24,844
ASVspooof 2019 LA eval	Unknown attacks (A07–A19, 11 unseen)	3.33%	0.994	71,237
ASVspooof 2021 LA eval	+ Telephony codecs (alaw, ulaw, gsm, opus, g722, pstn)	5.67%	0.984	148,176
ASVspooof 2021 DF eval	+ Media compression + 100+ unseen TTS/VC systems	22.95%	0.881	533,928

All four under the PRD’s 25% gate. Degradation is **monotonic and graceful** — the right signature for a working ML system. Erratic jumps would indicate overfitting or evaluation bugs.

Per-attack analysis on 2021 DF

Pattern matches published ASVspooof literature exactly:

Attack	EER	Notes
A09 (Griffin-Lim TTS)	0.79%	Trivially easy — published literature says the same
A13 (moment-matching VC)	3.96%	Easy
A11 (GAN TTS)	8.90%	Mid-range
A17 (VAE-VC + waveform filtering)	31.82%	Hardest TTS/VC by paper consensus
A18 (i-vector/PLDA VC + DNN glottal vocoder)	40.93%	Hardest published anti-spoofing test case

The model fails where the field fails (A17, A18) and trivializes what the field finds easy (A09). That's a signal of legitimate generalization, not a bug.

Per-compression robustness on 2021 DF (PRD FR-12 fairness slice)

Compression	EER
low_ogg	21.96%
oggm4a	22.10%
high_ogg	22.66%
nocodec	23.19%
mp3m4a	23.19%
high_mp3	23.19%
high_m4a	23.27%
low_m4a	23.32%
low_mp3	23.63%

Spread is only 1.67 percentage points across all 9 compression conditions. The model isn't fragile to compression — it's learning the spoof signal itself, not compression artifacts. Production-relevant: we don't need to know what codec hit the audio to predict performance.

2. Addressing last sync's "too good to be true" concern

You raised the right question. Documenting the reasoning here so it's auditable.

Why the result lands where it does

1. **AASIST is a published reference architecture** (Jung et al., ICASSP 2022, arXiv:2110.01200). The paper reports 0.83% EER on this dataset; I got 0.90% with 25 epochs vs the paper's 100 epochs (no SWA). I reproduced a published result — I didn't invent a new number. If I'd gotten dramatically worse, *that* would indicate a broken pipeline.
2. **Splits are properly disjoint by ASVspoof's design.** 2019 LA has 20 train speakers, 20 different dev speakers, and 48 different eval speakers. I trained only on the train split. The 0.90% is on the dev split — held-out data the model never saw during training. No data leakage.

3. **No caching at inference time.** Every prediction is recomputed fresh from raw audio through the model. There is no pretrained weight cache — AASIST was trained end-to-end from random initialization, no transfer learning involved. There is no audio cache — files are loaded from disk and processed live for each scoring.
4. **The degradation curve validates the result.** A model that magically does well on easy data and falls apart on hard data would be concerning. A model that degrades smoothly from 0.90% → 22.95% across increasing difficulty is behaving correctly. Each step on the curve adds one new dimension of difficulty, and EER rises proportionally.
5. **Per-attack pattern is consistent with the field.** A17 and A18 are the hardest in my evaluation. They are also the hardest in every published ASVspoof analysis. Failing where the field fails (rather than where the field succeeds) is the right signature.

What I'd worry about if it weren't right

- If splits weren't disjoint → I'd disclose it
- If I used pretrained weights → I'd disclose it (I didn't)
- If the curve was non-monotonic or stayed flat across difficulty → that would suggest something is wrong (it does the right thing)
- If per-attack patterns were random rather than literature-aligned → that would suggest spurious learning (they're not)

The full pipeline is reproducible from `train_aasist.py` with `seed=1234`. Every number in this memo can be re-derived.

3. Engineering roadblocks this sprint

Honest engineering reporting — three roadblocks worth flagging because they shaped some of the choices I made.

Roadblock 1: UVA HPC instability (Thursday – Sunday)

Rivanna had partial storage outages affecting login nodes and `/home` reads. OOD shells dropped frequently and conda activations failed silently due to a subtle `PATH` issue.

How I resolved: Switched to terminal SSH via UVA VPN for stability, and got disciplined about checking `which python` after every `conda activate` to catch the broken-activation state.

Impact on results: None on final numbers; some work shifted later in the week.

Roadblock 2: ASVspoof 2021 audio decode failures (~44% file skip rate initially)

Initial 2021 LA evaluation showed 44% of files failing to decode via `libsndfile` (used by both `torchaudio` and `soundfile`). Original error message was misleading (`<exception str() failed>`) — masked the real cause.

Root cause: 2021 LA audio includes FLAC variants `libsndfile` can't decode. Compounded because the system-wide `ffmpeg` on Rivanna is broken (missing `libvmaf.so.0`).

How I resolved: Built a 3-tier audio loader (`torchaudio` → `soundfile` → `ffmpeg` subprocess) and installed a working `ffmpeg` into my conda env via conda-forge. Coverage went from 56% to 100% on the re-run.

Impact on results: The originally-reported 5.71% on 2021 LA had a 44% skip-rate bias. The corrected **5.67% on full coverage** is the trustworthy number. Per-codec breakdown actually became more consistent after the fix — the “none” (clean) codec dropped from being the worst to being mid-range, matching intuition.

Roadblock 3: Slurm A100 queue saturation (multi-day wait at peak)

A100 queue depth hit ~2.8 days at one point — wall-time risk vs the Friday sponsor sync.

How I resolved: Switched to NVIDIA A6000 for the training run. For AASIST's 297K-parameter model, A6000 throughput was actually fine — model size dominates wall time, not GPU specifics. Full 25-epoch training finished in **3.3 hours wall time**.

Tradeoff documented: A6000 vs A100 for a small model is essentially a no-op decision — same quality, ~15% slower throughput, much shorter queue. For larger models (like Wave2Vec 2.0, which has ~95M params), this tradeoff would be less favorable.

4. Key decisions made + tradeoffs

Per your guidance, here are the major decisions I made this sprint in a 4-column format. Going forward, I'm setting up a `/docs/decisions/` folder in my work area where future decisions will be written **before** implementation.

Decision driver	Choices considered	Actual decision	Tradeoffs / implications
Which dataset to train on	ASVspoof 2019 LA, 2021 LA, 2021 DF	2019 LA	2021 sets are eval-only (no training data); 2019 LA is the standard training corpus. Lost

Decision driver	Choices considered	Actual decision	Tradeoffs / implications
			nothing; gained the standard reproducible baseline.
Which GPU for training	A100 (40/80GB), A6000, V100	A6000	A100 queue was 2.8 days at peak, deadline risk. A6000 queue: 15 min. For AASIST's small size, no quality difference. Lost: ~15% throughput. Gained: 3 hours wall time and no missed deadline.
Training epoch count	100 (paper standard), 50, 25	25 epochs	Cosine LR with T_max=25 . Result: 0.90% dev EER (paper reports 0.83% at 100 epochs). Lost: ~0.07 pp accuracy. Gained: 3.3 hr wall time vs ~13 hr. Acceptable for this sprint; will reconsider for PR1 final.
Label convention	bonafide=0 + spoof=1 vs bonafide=1 + spoof=0 (AASIST paper)	bonafide=0, spoof=1	Standard convention across most public anti-spoofing implementations. Required flipping class weights from paper's [0.1, 0.9] to my [0.9, 0.1] (now upweighting the minority bonafide class). No quality impact.
Augmentation for training	RawBoost vs no augmentation	No augmentation (v1)	RawBoost adds ~1 day to wire in. Wanted a clean baseline first before adding augmentation. Tradeoff: 2021 cross-domain numbers higher than they could be (22.95% on DF vs published ~15% <i>with</i> RawBoost). RawBoost is

Decision driver	Choices considered	Actual decision	Tradeoffs / implications
			on next sprint's roadmap.
Eval coverage strategy	Skip broken files vs robust 3-tier loader	Robust 3-tier loader	44% of 2021 LA files initially failed <code>libsndfile</code> . Building the <code>ffmpeg</code> fallback added ~half a day. Gained: 100% coverage, unbiased numbers, reusable loader for 2021 DF.
Per-utterance evaluation granularity	Overall EER only vs per-attack + per-codec breakdowns	Full breakdowns	Per-attack + per-codec output satisfies PRD FR-12 (fairness across acoustic conditions) and gives sponsor-presentable detail. Marginal extra compute time.

5. Explainability direction — SHAP

Picking up on your point that **Shapley values are the more widely-used standard** — that's consistent with the iWAX framework we previously agreed on. My plan for the iWAX explainability layer is:

1. Extract AASIST's penultimate-layer embeddings on the training set
2. Train an XGBoost classifier on those embeddings
3. Compute **SHAP values** (`shap.TreeExplainer` is the native tool for XGBoost) to attribute spoof predictions to specific frequency bands and time segments
4. Combine SHAP-derived attributions with AASIST's built-in graph attention weights for a complete explainability artifact

I will not use LIME or attention-only methods — SHAP is the target. Scaffolding starts next sprint.

6. What I'm doing next sprint (Jun 5 – Jun 19)

On my AASIST branch:

Week	Milestone
Jun 5 – Jun 14	RawBoost augmentation re-training to close the 2021 DF gap (target: 22.95% → ~12-15%). Begin iWAX scaffold: AASIST embeddings extraction + XGBoost head + SHAP integration.
Jun 14 – Jun 19	Per-utterance latency profiling for PRD NFR (≤ 4 min target; expecting < 5 ms on A100 given model size). Final iWAX prototype. PR1 polish.

7. Process changes I'm adopting based on your guidance

Acknowledging the feedback last sync, here's what I'm changing in how I work:

1. **Decision docs written *before* implementation.** New `/docs/decisions/` folder in my work area using the 4-column format above. Backfilling the decisions I made this sprint.
2. **Weekly accomplishment summary** sent before each sponsor sync. This memo is the first instance.
3. **Documentation with author tags and change logs.** Every doc I write now starts with author tag at top + change log section.
4. **Documented "why" alongside results** rather than just reporting numbers. This memo's Section 2 is a deliberate example.

8. Open questions for sponsors

1. **iWAX framework — still the right direction?** Confirming you'd like SHAP-on-XGBoost over AASIST embeddings. If you'd prefer a different explainability approach, easier to course-correct now than at PR1.
2. **AWS credits status?** No blocker — UVA HPC is sufficient. Would help me scope a credible cloud-mirror demo for the production-direction story if credits land.

Reproducibility

Everything described is on Rivanna and reproducible. All my run artifacts have group-readable permissions for the rest of the team to audit.

- **Trained checkpoint:** `/scratch/mhq8ka/aasist/outputs/runs/aasist_fast_13608120/best.pt`

- **Training script:** `/scratch/mhq8ka/aasist/code/aasist_branch/train_aasist.py`
- **Eval scripts:** `/scratch/mhq8ka/aasist/code/aasist_branch/eval_aasist*.py`
- **Per-utterance predictions:** `/scratch/mhq8ka/aasist/outputs/eval/{2019_LA, 2021_LA, 2021_DF}_eval/predictions.csv`
- **Random seed:** 1234, `cudnn.deterministic=True` , per PRD reproducibility NFR

Thanks for the feedback last sync — both the “too good to be true” rigor check and the process guidance pushed the work forward.

— **Arnav Jain** UVA MSDS Capstone — AASIST Modeling Branch